



Entornos de Desarrollo

***UT5- Diseño orientado a
objetos. Elaboración de
diagramas estructurales***

Programación Orientada a Objetos

La Programación orientada a objetos (POO), al contrario que la programación estructurada centrada en procedimientos, se centra en simular los elementos de la realidad asociada al problema de la forma más cercana posible (objetos)

Los objetos están formados por un conjunto de atributos, que son los datos que le caracterizan y un conjunto de operaciones que definen su comportamiento. Las operaciones asociadas a un objeto actúan sobre sus atributos para modificar su estado. Cuando se indica a un objeto que ejecute una operación determinada se dice que se le pasa un mensaje

COCHE

- Objeto:
 - Tiene propiedades (atributos):
 - Color
 - Peso
 - Alto
 - Largo
 - Tiene un comportamiento (¿Qué es capaz de hacer?):
 - Arrancar
 - Frenar
 - Girar
 - Acelerar

<https://www.pildorasinformaticas.es/course/course-poo-programacion-orientada-a-objetos/>



Programación Orientada a Objetos

Conceptos Básicos

Objeto

Clase

Instancia

Abstracción

Modularización

Encapsulamiento

Herencia

Polimorfismo

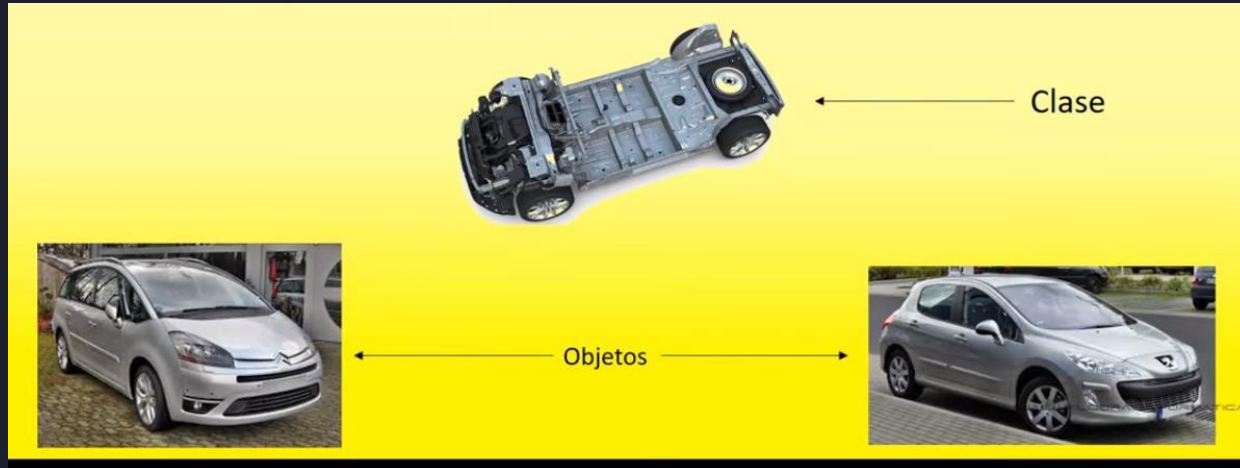
Programación Orientada a Objetos

Conceptos Básicos

Objetos: formado por atributos (propiedades) y operaciones (comportamientos)

Clase: características comunes de un grupo de objetos

Instancia: ejemplar perteneciente a una clase





Programación Orientada a Objetos

Conceptos Básicos

- Objeto:
 - Accediendo a propiedades de objeto desde código (pseudocódigo):
 - `miCoche.color="rojo";`
 - `miCoche.peso=1500;`
 - `miCoche.ancho=2000;`
 - `miCoche.alto=900;`
 - Accediendo a comportamiento de objeto desde código (pseudocódigo):
 - `miCoche.arranca();`
 - `miCoche.frena();`
 - `miCoche.gira();`
 - `miCoche.acelera();`

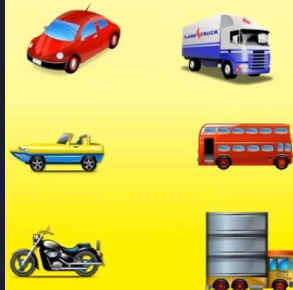
Programación Orientada a Objetos

Conceptos Básicos

Abstracción: obtener las características y comportamientos similares de un conjunto de objetos para crear todas las clases necesarias para modelar el problema a resolver

Encapsulación: la idea es evitar el acceso a datos por medios no permitidos. El estado de los objetos sólo debería poder ser modificado desde métodos de la propia clase

- Para reutilizar código en caso de crear objetos similares



¿Qué características en común tienen todos los objetos?

Marca

Modelo

¿Qué comportamientos en común tienen todos los objetos?

Arrancan

Aceleran

Frenan



Programación Orientada a Objetos

Modularidad: la idea es dividir una aplicación en partes más pequeñas (módulos), que sean lo más independientes posibles. En POO los objetos son los módulos más básicos del sistema.

Polimorfismo: capacidad que tienen los objetos de una clase para ofrecer respuestas distintas e independientes en función de los parámetros

Herencia: Relación que se establece entre objetos en los que unos utilizan las propiedades y comportamientos de otros formando una jerarquía. Los objetos heredan las propiedades y el comportamiento de todas las clases a las que pertenecen



UML

UML (*Unified Modeling Language* o *Lenguaje Unificado de Modelado*) es un conjunto de herramientas que permite modelar, construir y documentar los elementos que forman un sistema software orientado a objetos

Permite visualizar el trabajo en esquemas o diagramas estandarizados denominados modelos que representan el sistema desde diferentes perspectivas

También se pueden especificar todas las decisiones de análisis, diseño e implementación, construyéndose modelos precisos, no ambiguos y completos

UML

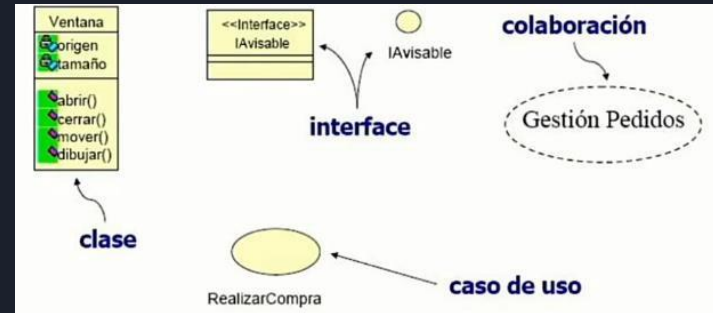
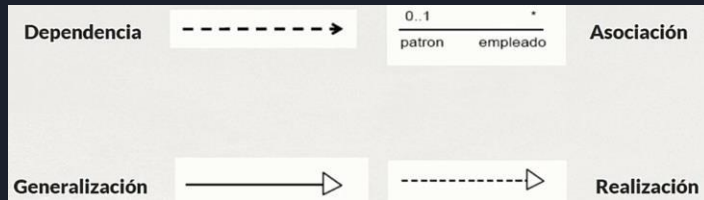
UML es un lenguaje (visual), no una metodología

Sus bloques de construcción básicos son:

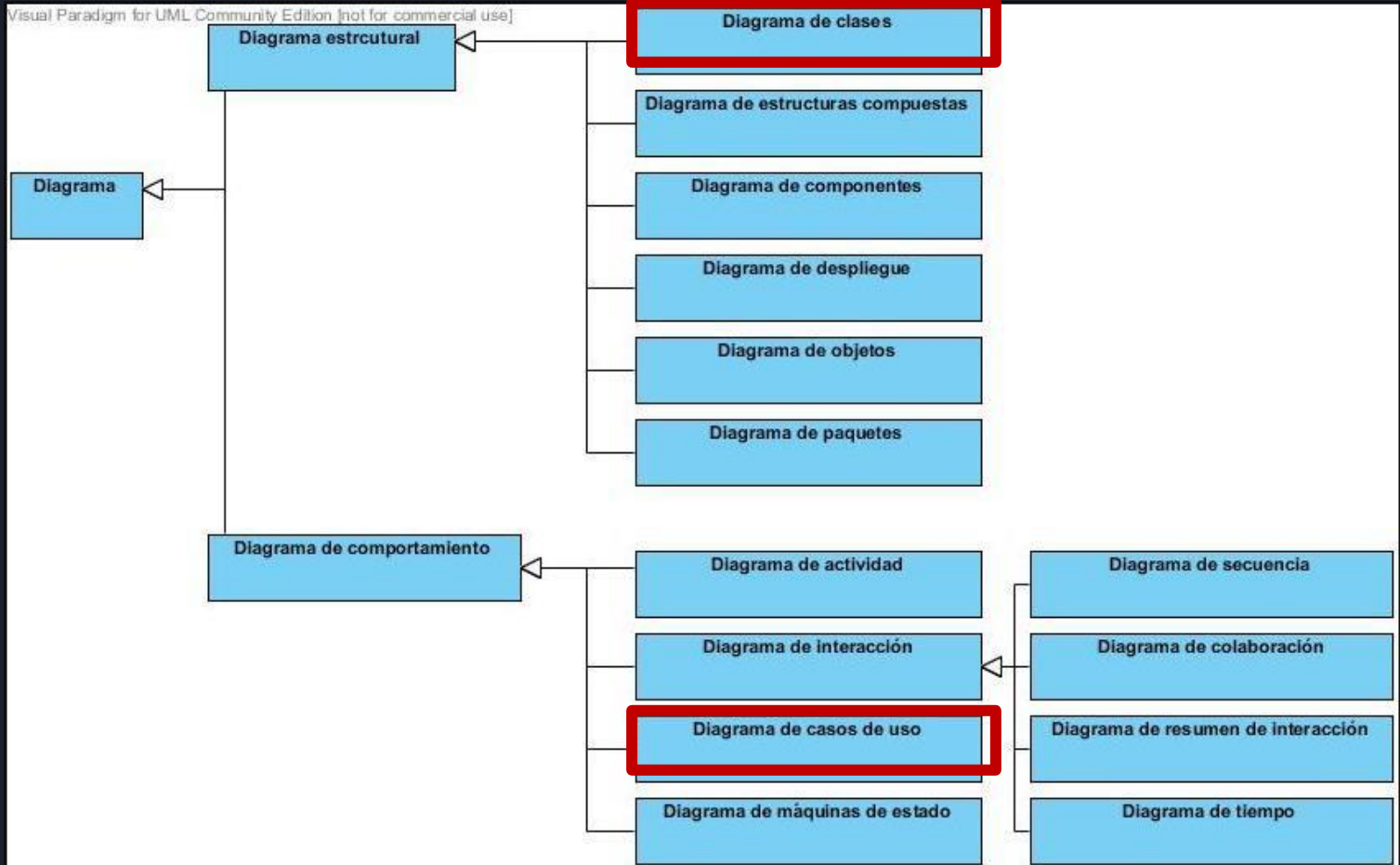
Elementos

Relaciones (conectan los elementos)

Diagramas



UML – Tipos de Diagramas





UML - Clasificación

Los diagramas UML se clasifican en:

***Diagramas estructurales:** Representan la visión **estática** del sistema. Especifican clases y objetos y como se distribuyen físicamente en el sistema

Diagramas de comportamiento: Muestran la conducta **en tiempo de ejecución** del sistema, tanto desde el punto de vista del sistema completo como de las instancias u objetos que lo integran



UML - Diagramas

***Diagrama de clases:** (*De estructura*) está formado por un conjunto de clases y sus relaciones

Diagrama de casos de uso: (*De comportamiento*)

Representa las acciones a realizar en el sistema desde el punto de vista de los usuarios. En él se representan las acciones, los usuarios y las relaciones entre ellos. Sirven para especificar la funcionalidad y el comportamiento de un sistema mediante su interacción con los usuarios y/u otros sistemas

UML - EJEMPLOS

Diagrama de clases

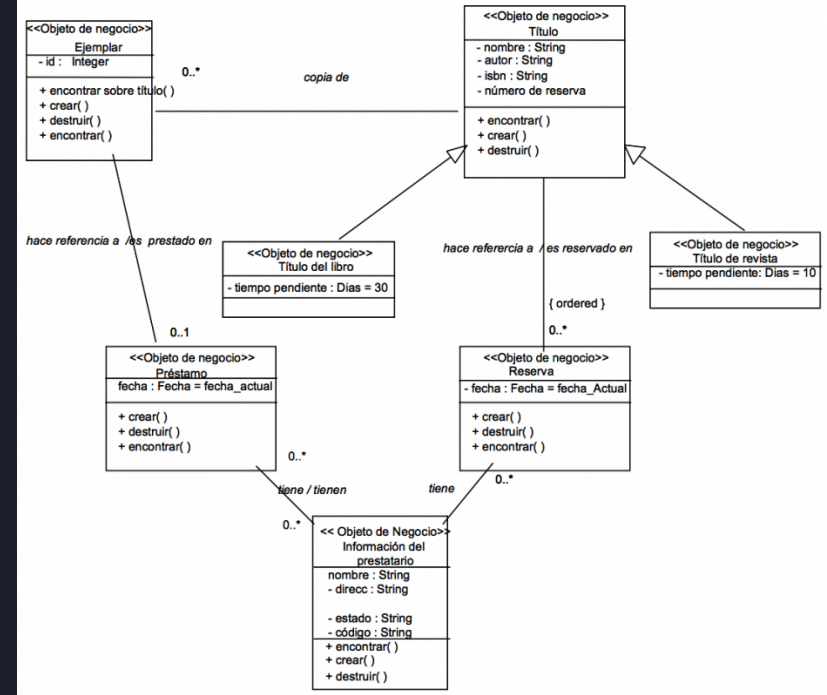
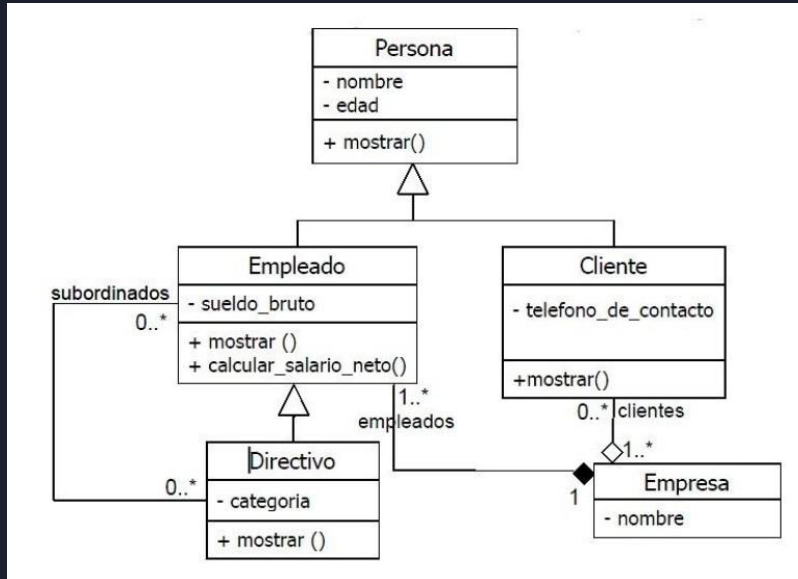
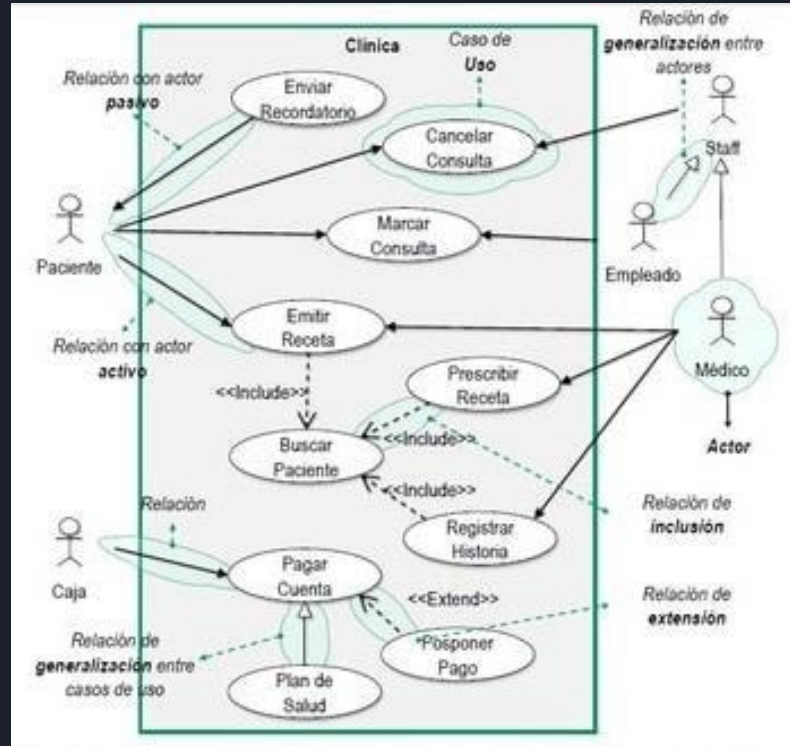


Diagrama de casos de uso





UML – HERRAMIENTAS IDEs

EasyUML: (Netbeans) ofrece un conjunto de notaciones textuales simples que después son traducidas a los correspondientes modelos UML gráficos. Se puede utilizar para documentar una vez creado el código o, para crear el código a partir de la documentación

Ha sido eliminado de la página oficial de plugins de Apache y las versiones que existen dan problemas de instalación en el IDE



UML – HERRAMIENTAS IDEs

PlantUML: al igual que EasyUML, ofrece un conjunto de notaciones textuales simples que después son traducidas a los correspondientes modelos UML gráficos. No es muy estético, pero se puede utilizar en VS Code, IntelliJ, Netbeans, entre otros

Página Oficial: <https://plantuml.com/>

Página de descarga del Plugin para Netbeans:
<https://plugins.netbeans.apache.org/>

UML - HERRAMIENTAS

Visual Paradigm UML: Posible integración con NetBeans.
Generación de código y documentación. Ingeniería inversa.
Es utilizado en los contenidos del CAMPUS

The screenshot displays the Visual Paradigm Enterprise (Evaluation Copy) interface. The top menu bar includes Dash, Project, ITSM, Agile, Diagram, View, Team, Tools, Modeling, Window, and Help. Below the menu is a toolbar with icons for New, Open, Save, Close, Print, Import, Export, Referenced Project, Properties, and Community Circle. The main workspace is titled "Welcome to Visual Paradigm Enterprise" and features a navigation pane on the left with sections for Diagram Navigator, Model Explorer, and Property. The central area is divided into three main sections: "Architecture Frameworks", "EA Process (Guide-Through / JIT Process / Canvas)", and "EA Modeling".

Architecture Frameworks
Create architecture views with traceability maintained among views. Manage and access the views through the grid interface.

Framework	Icon
DoDAF	+
NAF	+
MODAF	+

EA Process (Guide-Through / JIT Process / Canvas)
Actionable process map / canvas that guides you through the planning, design and development of EA and business processes.

Process	Icon
TOGAF ADM Guide-Through Process	+
TOGAF ADM Just-in-Time Process Map	+
Business Process Reengineering	+
Business Motivation Model	+

EA Modeling
Visualize EA and business processes with ArchiMate and BPMN business process diagrams.

Modeling Language	Icon
ArchiMate	+
BPMN	+

At the bottom, there are two promotional banners. The left one says "Store and share your design on your cloud workspace" with a link icon. The right one says "Jump start your design with diagram templates" with a link icon.



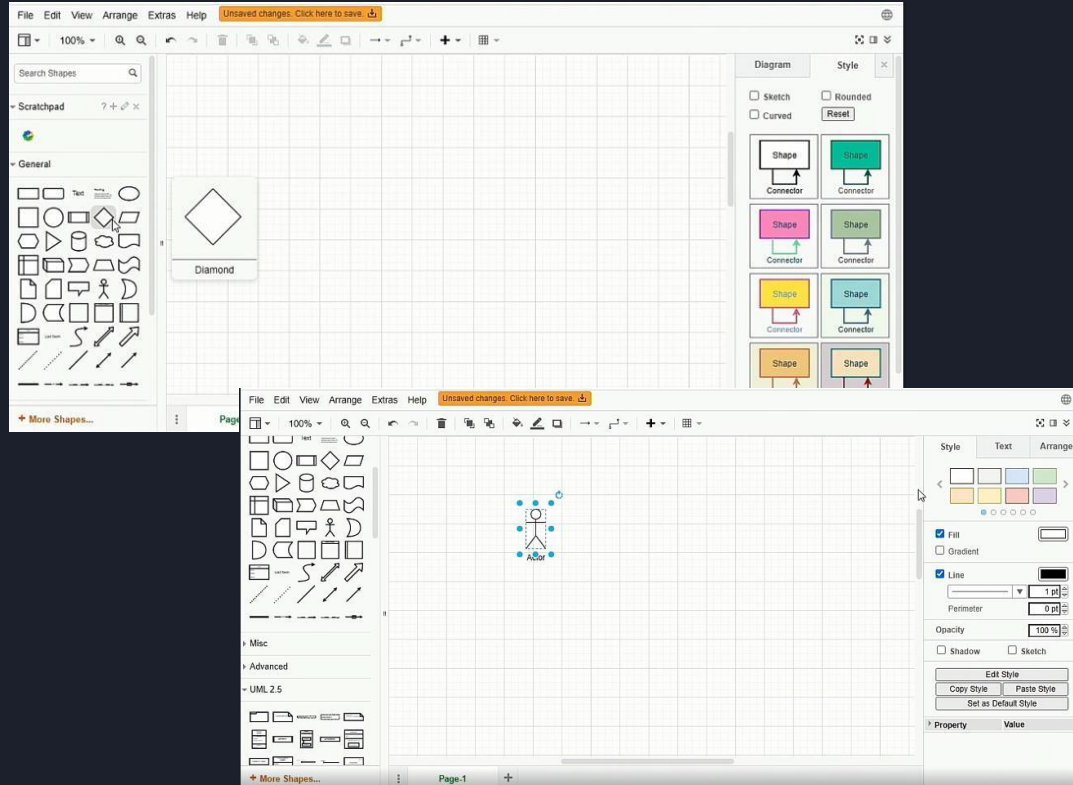
UML - HERRAMIENTAS

UMLet: gratuito, no requiere instalación ni registro. Tiene versión en línea UMLetino. Es utilizado en los contenidos del CAMPUS

LucidChart: crear diagramas UML, canvas, redes de datos. Requiere registro

UML - HERRAMIENTAS

DRAW.IO: es online, no requiere instalación, ni registrarse





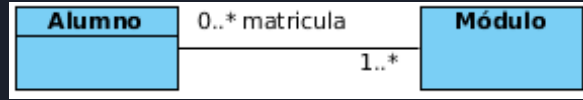
UML DIAGRAMAS DE CLASES

Relaciones

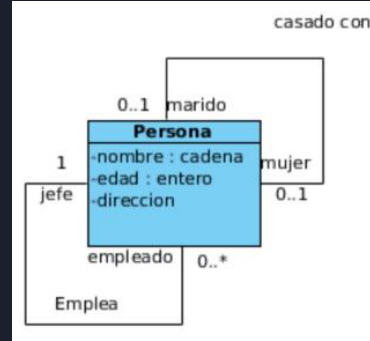
- **Asociación:** este tipo de relación es el más común y se utiliza para representar dependencia semántica.
- **Agregación:** los elementos parte pueden existir sin el elemento contenedor y no son propiedad suya.
- **Composición:** los elementos que forman parte no tienen sentido de existencia cuando el primero no existe.
- **Dependencia:** Se utiliza este tipo de relación para representar que una clase requiere de otra para ofrecer sus funcionalidades.
- **Herencia:** una clase (clase hija o subclase) reciba los atributos y métodos de otra clase (clase padre o superclase).

UML DIAGRAMAS DE CLASES

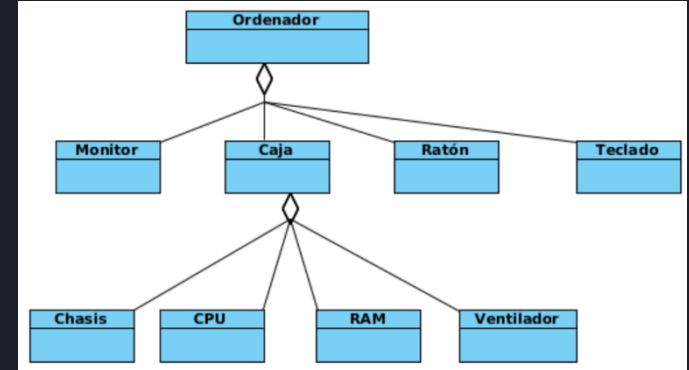
Relaciones



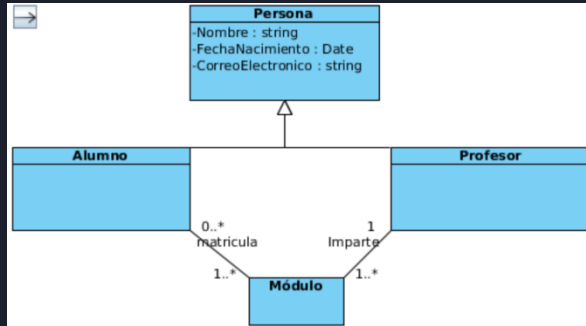
Asociación



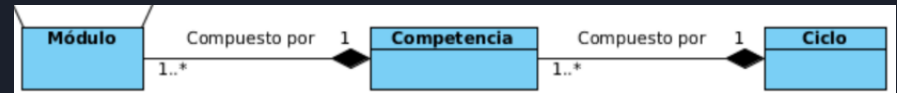
Unarias



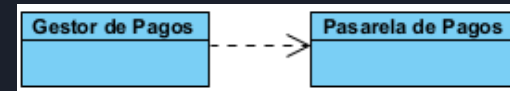
Agregación



Herencia



Composición



Dependencia

UML DIAGRAMAS DE CLASES

Relaciones

Significado de las cardinalidades.

Cardinalidad	Significado
1	Uno y sólo uno
0..1	Cero o uno
N..M	Desde N hasta M
*	Cero o varios
0..*	Cero o varios
1..*	Uno o varios (al menos uno)